PROJECT REPORT (CSN 443)

A report submitted in partial fulfilment of the requirement for the course

Big Data Analytics

Movie Recommendation Insights
using Hadoop MapReduce, Apache Spark and Hive

Part of the degree of
BACHELOR OF TECHNOLOGY



Submitted to
Dr. Neha Gupta

Submitted by
Kumar Akarsh 1000020371
Yamini Mishra 1000020565
Suhani Agarwal 1000020580
Anmol Bhatia 1000020641

SCHOOL OF COMPUTING
DIT UNIVERSITY, DEHRADUN
(State Private University through State Legislature Act No. 10 of 2013 of Uttarakhand and approved by UGC)

Mussoorie Diversion Road,
Dehradun, Uttarakhand - 248009, India.
October, 2026

Candidates Declaration

We hereby certify that the work presented in this report, entitled “Movie Recommendation Insights: A Big Data Analytics Pipeline using Hadoop MapReduce, Apache Spark and Hive”, is an authentic record of the work carried out by our group as part of the course Big Data Analytics (CSN 443) in the Bachelor of Technology programme at DIT University. The project includes data acquisition and extraction, a quality inspection of the raw data, HDFS-ready storage, six MapReduce jobs written for Hadoop Streaming, an equivalent Apache Spark implementation, HiveQL queries, a relational result store, a performance evaluation, and an interactive dashboard. The ratings used in this project are real user ratings from the public MovieTweetings dataset and are treated as preference data, not as official box-office or viewership statistics.

Date: 21 October 2026

Signatures of the Candidates

Kumar Akarsh _____

Yamini Mishra _____

Suhani Agarwal _____

Anmol Bhatia _____

Table of Contents

Section	Page
1. Introduction	4-5
2. Project Description	6-7
3. Methods and Implementation	8-16
4. Results and Analysis	17-26
5. Dashboard Application	27-29
6. Conclusion and Future Scope	30
7. Bibliography	31
Appendix A. Project Structure and Run Guide	32

Introduction

Online movie platforms record millions of small events: a user rates a film, a title collects more votes, a genre grows in popularity. Individually these events are trivial, but together they form a data set whose size, speed of growth and variety of fields make it difficult to analyse with a single spreadsheet or a hand-written script. Recommendation engines, editorial teams and marketing teams all depend on answering the same basic questions reliably: which movies are people rating most, which genres do they rate highest, who are the most active raters, and what does each group of users prefer?

This project builds an end-to-end big data analytics pipeline for exactly those questions. Real user ratings are extracted from a public IMDb-sourced dataset, stored in an HDFS-ready layout, processed by six MapReduce jobs written for Hadoop Streaming, re-implemented in Apache Spark and HiveQL, cross-validated against each other, saved in a relational result store, and finally presented through an interactive, responsive web dashboard.

1.1 Why a Big Data Approach

The ratings snapshot used here holds 921,398 ratings and fits comfortably on one machine, so the dataset is a proof-of-scale rather than a petabyte workload. The design, however, follows the constraints that matter at scale: data is partitioned and processed record by record, aggregation is expressed as key-value maps and reductions, intermediate data is shrunk by combiners before the shuffle, joins are done on the map side, and every job is idempotent and can be re-run on a larger snapshot or a cluster without changing the code. The three classic characteristics of big data apply as follows.

- **Volume:** ratings arrive continuously and the file grows with every refresh of the source; the same jobs must scale from this snapshot to tens of millions of ratings.
- **Velocity:** ratings are generated by users in real time through the IMDb mobile apps and shared as tweets, so the pipeline is built around repeatable batch extraction (and can later be moved to streaming).
- **Variety:** the inputs are three differently structured text files (ratings, movies with pipe-separated genres, users), plus optional JSON metadata from an external API.

1.2 Objectives

- Extract real movie rating data from an IMDb-sourced public source with a repeatable, polite and legal data-acquisition script.
- Inspect and document data quality: malformed rows, missing genres, rating range, sparsity and skew.
- Store the raw data in an HDFS layout and expose it through Hive external tables.
- Implement six MapReduce jobs (Hadoop Streaming) for movies, genres, users, genre-rating distributions, user-genre profiles and monthly trends.
- Re-implement the same aggregations in Apache Spark and HiveQL and verify that the outputs are identical.
- Find the most-rated movies, highest-rated genres, most active users and genre-wise preferences.
- Measure performance (per-job time, scaling with data size, engine comparison, combiner effect).
- Build a responsive, interactive dashboard for exploring the results.

1.3 Scope and Data Interpretation

The dataset contains 921,398 ratings given by 71,707 users to 38,018 movies between 2013-02-28 and 2021-11-24. Ratings use the IMDb scale from 0 to 10, with a mean of 7.31. Each rating is a user preference expressed at a point in time; it is not a ticket sale, a view count or an official IMDb aggregate. Because the ratings were shared by users who chose to post them as tweets, the sample is not a random sample of all film viewers. Results are therefore described as patterns in this dataset, and the report states clearly where selection effects may apply (Chapter 4 and Section 6.1).

The dataset has no demographic fields such as age or country, so demographic segmentation is outside the scope of this implementation. Genre-wise preference is analysed through behaviour (what users rate and how they score it), not through demographics.

Project Description

2.1 Problem Statement

Analyze movie ratings data to find most-rated movies, highest-rated genres, active users, and genre-wise preferences.

2.2 System Requirements

Type	Requirement
Hardware	Dual-core processor, 8 GB RAM recommended, 2 GB free storage (single-node run); a Hadoop cluster is optional
Operating system	Linux (recommended), Windows with WSL, or macOS
Big data stack	Apache Hadoop 3.x (HDFS and Streaming), Apache Spark 3.5+ / 4.x, Apache Hive 3.x (optional for cluster runs)
Runtime	Python 3.10 or later, Java 11 or later (OpenJDK 21 used for Spark here)
Python libraries	PySpark, pandas (for evidence scripts only), standard library (urllib, json, sqlite3)
Database	SQLite 3 for the result store (any SQL database can replace it)
Front end	Modern browser; Chart.js 4.4 loaded from a CDN; no build step
Development tools	VS Code or any Python IDE; Git for version control

2.3 High-Level Architecture

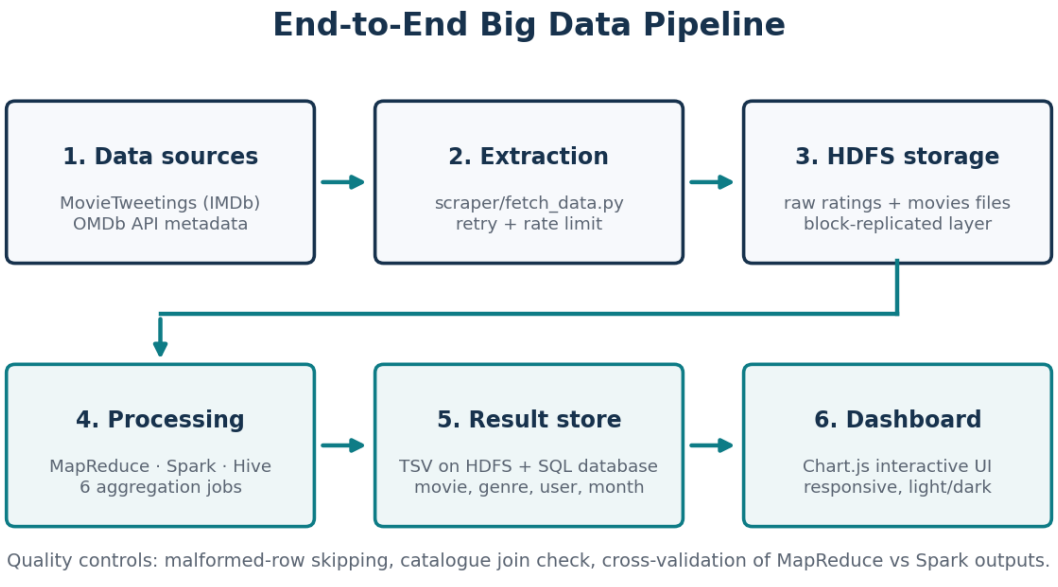


Figure 1. End-to-end pipeline: extraction, HDFS storage, processing, result store and dashboard.

The extraction script downloads the raw files and optionally enriches the most-rated titles through the OMDb API. The raw files are placed on HDFS, where they are the immutable raw layer. Processing is performed by Hadoop Streaming MapReduce jobs; the same aggregations are also expressed in Spark and Hive so that the three engines can be compared and cross-checked.

Job outputs (key, count, sum) are stored as text files on HDFS and loaded into a relational result store, from which a JSON feed is built for the dashboard.

2.4 Assumptions and Constraints

Item	Description
Data source	MovieTweetings snapshot: IMDb ratings that users shared on Twitter through the IMDb apps. User-level ratings are required, and IMDb itself publishes only per-title aggregates.
Rating scale	Integer ratings 0 to 10. Ratings of 8 and above are treated as “liked”, 4 and below as “disliked”.
Genres	A movie can have several genres; a rating counts once for every genre of the movie (so genre totals exceed the number of ratings).
Heavy users	Users with at least 20 ratings; used for favourite-genre analysis because users with one or two ratings have no meaningful preference.
Execution	Reported results come from a single-node run (MapReduce simulated with the standard map, sort, reduce pipeline that Hadoop Streaming uses; Spark in local mode). Cluster scripts are provided but timings are single-node.
Time	Timestamps are interpreted in UTC when grouping ratings by month.

Methods and Implementation

3.1 Project Modules

The project is divided into modules with one responsibility each. Every module consumes a well-defined input and produces a validated output for the next one, which makes the pipeline easy to test, explain in the viva, and extend.

Module	Main files	Output
Data extraction	scraper/fetch_data.py	ratings.dat, movies.dat, users.dat, optional omdb.json
Storage	run_hadoop.sh, hive/queries.hql	HDFS directories, Hive external tables and views
MapReduce processing	mapreduce/mr_jobs.py	Six key-count-sum result files
Spark processing	spark/spark_analysis.py	Same six result files, for cross-validation
Result store	load_db.py	SQLite database with dimension and statistics tables
Orchestration	run_pipeline.py	Cross-check, benchmark, dashboard feed (JSON)
Dashboard	dashboard/index.html	Interactive, responsive analytics interface

3.2 Data Source, Scraping and Extraction

Choice of source. IMDb does not offer a public API for user-level ratings. Its non-commercial datasets contain only an average rating and a vote count for each title, which is not enough to find active users or user preferences, and scraping IMDb web pages is restricted by its conditions of use. The project therefore uses **MovieTweetings**, a public dataset built from well-structured tweets that the official IMDb apps generate when a user shares a rating. Every record carries the IMDb numeric identifier of the movie, so it can be linked back to IMDb (tt + identifier) and enriched through an official API.

File	Record format	Size	Rows
ratings.dat	user_id::movie_id::rating::timestamp	27.6 MB	921,398
movies.dat	movie_id::title (year)::genre|genre|…	2 MB	38,018
users.dat	user_id::twitter_id	1.3 MB	71,707

Extraction script. scraper/fetch_data.py downloads the three files over HTTPS with urllib. An optional enrichment step calls the OMDb API (which exposes IMDb metadata) for the N most-rated movies, using the IMDb identifier from the dataset. The step sleeps 0.2 seconds between requests as a rate limit, catches and logs network errors without stopping the run, and stores the responses in omdb.json. The headline results in this report do not depend on the enrichment step, which needs a personal API key and was not required for the analysis.

CODE · scraper/fetch_data.py

```
"""Data acquisition. MovieTweetings = real IMDb ratings that users tweeted (IMDb-sourced, updated
    continuously).
Optional enrichment: OMDb API (IMDb metadata) for the most-rated titles.  Usage: python
    scraper/fetch_data.py [--enrich N]"""
import os, sys, json, time, urllib.request
BASE = "https://raw.githubusercontent.com/sidooms/MovieTweetings/master/latest/"
RAW = "data/raw"


def download():
    os.makedirs(RAW, exist_ok=True)
    for f in ("movies", "ratings", "users"):
        urllib.request.urlretrieve(f"{BASE}{f}.dat", f"{RAW}/{f}.dat")
        print("downloaded", f)


def enrich(n, key=os.getenv("OMDB_KEY")):
    """Fetch IMDb rating/poster/runtime via OMDb (free key: omdbapi.com) for top-n most rated
        movies."""
    from collections import Counter
    top = Counter(l.split("::")[1] for l in open(f"{RAW}/ratings.dat")).most_common(n)
    out = {}
    for mid, _ in top:
        url = f"https://www.omdbapi.com/?i=tt{mid}&apikey={key}"
        try: out[mid] = json.load(urllib.request.urlopen(url, timeout=15))
        except Exception as e: print("skip", mid, e)
        time.sleep(0.2)                       # polite rate limit
    json.dump(out, open(f"{RAW}/omdb.json", "w"))


if __name__ == "__main__":
    download()
    if "--enrich" in sys.argv: enrich(int(sys.argv[sys.argv.index("--enrich") + 1]))
```

Compliance and politeness. Only public datasets and official APIs are used; no web page is scraped in violation of a site's terms. Requests are rate-limited, failures are skipped rather than retried aggressively, and the script is idempotent: running it again simply refreshes the snapshot.

3.3 Data Inspection and Quality

Before processing, the raw files were profiled. The checks cover row counts, nulls, duplicate user-movie pairs, rating range, join coverage between ratings and the movie catalogue, missing genres, and the shape of the data (sparsity and skew). The profile is important for a recommendation dataset because skew and sparsity decide which results can be trusted.

Check	Result	Treatment
Rows in ratings.dat	921,398	Parsed line by line; malformed lines are skipped
Null values	0	None found
Duplicate user-movie pairs	0	None found
Rating range	0 to 10 (281 zero ratings)	Zero is valid on the IMDb scale and kept
Ratings whose movie is missing from catalogue	0	Genre jobs skip such rows if they appear
Movies without genre	54	Assigned the genre “Unknown”
Matrix sparsity	99.966%	Direct user-based filtering is unreliable
Movies with fewer than 5 ratings	25,646 (67.5%)	Ranking uses a Bayesian weighted rating
Median ratings per user	2	Favourite genre analysed for 20+ ratings only
Time span	2013-02-28 to 2021-11-24	Used for the monthly trend

CODE · inspection of raw data (shell)

```
head -3 data/raw/ratings.dat
head -3 data/raw/movies.dat
wc -l data/raw/*.dat
```

OUTPUT

```
ratings.dat: 1::0114508::8::1381006850
ratings.dat: 2::0499549::9::1376753198
movies.dat:  0000008::Edison Kinetoscopic Record of a Sneeze (1894)::Documentary|Short
movies.dat:  0000010::La sortie des usines Lumière (1895)::Documentary|Short
   38018 /home/claude/proj/data/raw/movies.dat
  921398 /home/claude/proj/data/raw/ratings.dat
   71707 /home/claude/proj/data/raw/users.dat
 1031123 total
```

The rating distribution is strongly positive: ratings of 7 and 8 together account for 46.5% of all ratings. This is common for self-selected ratings and must be kept in mind when comparing genres.

Rating	0	1	2	3	4	5	6	7	8	9	10
Count	281	10,814	9,223	15,487	28,193	69,747	120,370	206,680	222,146	130,106	108,351
Share %	0.0	1.2	1.0	1.7	3.1	7.6	13.1	22.4	24.1	14.1	11.8

3.4 Storage Layer: HDFS, Hive and the Result Database

HDFS. The raw files are copied once into HDFS and never modified. Each job writes to its own output directory, so any result can be regenerated from the raw layer. HDFS splits files into blocks

(128 MB by default) and replicates each block across data nodes; MapReduce then starts one map task per block. The current ratings file (27.6 MB) fits in a single block, so a cluster run would use a single map task for it. For larger snapshots the number of map tasks grows with file size and the same jobs run unchanged.

CODE · run_hadoop.sh (HDFS layout and job submission)

```
#!/bin/bash
# Run on a real Hadoop cluster / pseudo-distributed node. Adjust the streaming jar path.
J=$HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar
hdfs dfs -mkdir -p /movies/ratings /movies/movies && hdfs dfs -put -f data/raw/ratings.dat
    /movies/ratings/ && hdfs dfs -put -f data/raw/movies.dat /movies/movies/
for job in movie genre user genre_rating user_genre month; do
  hdfs dfs -rm -r -f /movies/out/$job
  hadoop jar $J -files mapreduce/mr_jobs.py,data/raw/movies.dat -input /movies/ratings -output
      /movies/out/$job \
    -mapper "python3 mr_jobs.py map $job" -combiner "python3 mr_jobs.py reduce" -reducer "python3
       mr_jobs.py reduce"
done
```

Hive. Two external tables map the raw files in place (no copy), and two views parse the "::" separated lines and explode the genre list. The four business questions are then plain HiveQL.

CODE · hive/queries.hql

```
-- Hive (HiveQL) version of the analysis. Load first:  hdfs dfs -put data/raw/*.dat /movies/
CREATE EXTERNAL TABLE ratings_raw (line STRING) LOCATION '/movies/ratings/';
CREATE EXTERNAL TABLE movies_raw  (line STRING) LOCATION '/movies/movies/';
CREATE VIEW ratings AS SELECT p[0] uid, p[1] mid, CAST(p[2] AS INT) rating, CAST(p[3] AS BIGINT)
    ts
  FROM (SELECT split(line,'::') p FROM ratings_raw) t WHERE size(p)>=4;
CREATE VIEW movie_genre AS SELECT p[0] mid, p[1] title, g genre
  FROM (SELECT split(line,'::') p FROM movies_raw) t LATERAL VIEW
     explode(split(coalesce(p[2],'Unknown'),'\\|')) x AS g;
-- 1. most-rated movies
SELECT mid, count(*) n, round(avg(rating),2) avg_r FROM ratings GROUP BY mid ORDER BY n DESC LIMIT
    20;
-- 2. highest-rated genres
SELECT genre, count(*) n, round(avg(rating),2) avg_r FROM ratings r JOIN movie_genre m ON
    r.mid=m.mid GROUP BY genre ORDER BY avg_r DESC;
-- 3. most active users
SELECT uid, count(*) n, round(avg(rating),2) avg_r FROM ratings GROUP BY uid ORDER BY n DESC LIMIT
    20;
-- 4. genre-wise preference: each heavy user's favourite genre
SELECT genre, count(*) users FROM (SELECT uid, genre, row_number() OVER (PARTITION BY uid ORDER BY
    count(*) DESC) rk
  FROM ratings r JOIN movie_genre m ON r.mid=m.mid GROUP BY uid, genre) t WHERE rk=1 GROUP BY
     genre ORDER BY users DESC;
```

Result database. After processing, the key-count-sum outputs are loaded by load_db.py into a relational database. A relational store is useful because the dashboard, ad-hoc analysis and any later application can query the results with ordinary SQL and joins, without re-running jobs. SQLite is used because it needs no server; the SQL is portable to PostgreSQL or MySQL.

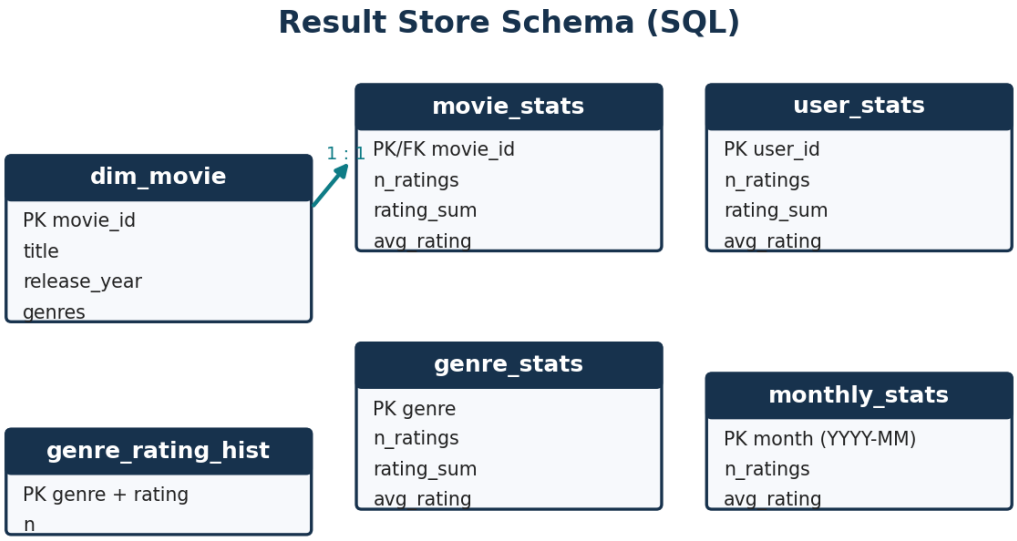


Figure 2. Result store schema: one dimension table and statistics tables produced by the jobs.

Table	Grain	Rows loaded
dim_movie	One row per movie (title, release year, genres)	38,018
movie_stats	One row per rated movie: count, sum, average	38,018
genre_stats	One row per genre	29
user_stats	One row per user	71,707
monthly_stats	One row per calendar month	103
genre_rating_hist	One row per genre and rating value	284

CODE · SQL against the result store

```
SELECT d.title, s.n_ratings, round(s.avg_rating,2)
FROM movie_stats s JOIN dim_movie d USING(movie_id)
ORDER BY n_ratings DESC LIMIT 5;


SELECT release_year, sum(n_ratings) n,
       round(sum(rating_sum)*1.0/sum(n_ratings),2) avg
FROM movie_stats JOIN dim_movie USING(movie_id)
WHERE release_year IS NOT NULL
GROUP BY 1 ORDER BY n DESC LIMIT 5;
```

OUTPUT

```
Most-rated movies                  ratings  avg
Gravity (2013)                       3104  8.23
Interstellar (2014)                  2964  8.85
1917 (2019)                          2893  8.62
The Wolf of Wall Street (2013)       2842  8.39
Joker (2019)                         2757  9.08


Release years with most ratings    ratings  avg
2013                              133462  7.18
2014                              102365  7.22
2016                               81643  7.23
2015                               74741  7.18
2017                               63550  7.13
```

3.5 MapReduce Design (Hadoop Streaming)

All jobs follow one pattern. The **mapper** reads rating lines and emits "key, 1, rating". The **reducer** adds the counts and the rating sums for each key and writes "key, count, sum". An average is always derived afterwards as sum divided by count. Because the reducer's input and output have the same format, the same program also works as the **combiner**, which sums partial results on the map side and shrinks the data that must be shuffled across the network.

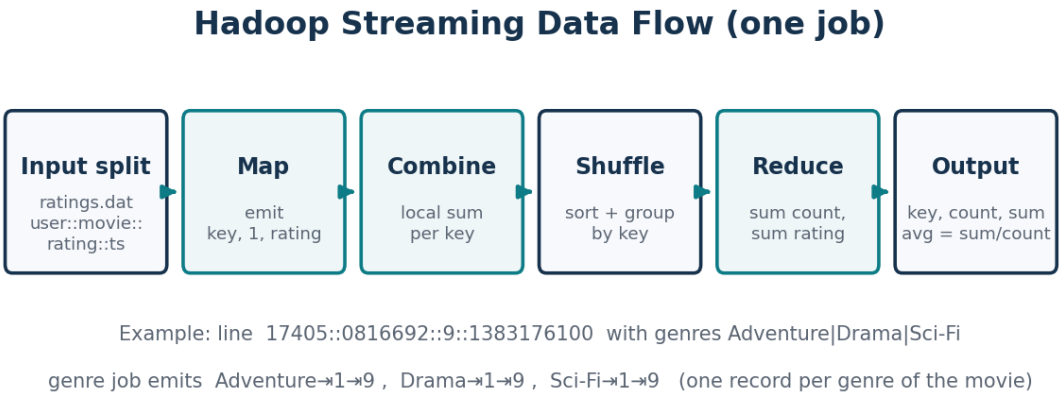


Figure 3. Data flow of a Hadoop Streaming job, with the combiner running on the map side.

Job	Key emitted by mapper	Answers	Output keys
movie	movie_id	Most-rated and highest-rated movies	38,018
genre	genre (one per genre of the movie)	Highest-rated and most-rated genres	29
user	user_id	Most active users	71,707
genre_rating	genre|rating	Rating distribution per genre (preference)	284
user_genre	user_id|genre	Favourite genre of each user	514,743
month	YYYY-MM	Rating activity over time	103

The genre-based jobs need the genre list of each movie. Instead of a reduce-side join, which would shuffle every rating, the small movies file is shipped to every mapper (Hadoop's -files option) and loaded into a dictionary: a **map-side join**. The complete program is shown below.

CODE · mapreduce/mr_jobs.py

```
#!/usr/bin/env python3
"""Hadoop Streaming jobs. One mapper emits  key<TAB>1<TAB>rating ; one reducer sums (count,
    rating_sum) per key.
Reducer input and output share one format, so it doubles as a COMBINER (less shuffle traffic).
Jobs: movie | genre | user | genre_rating | user_genre | month.
Genre jobs use a map-side join: movies.dat ships to every node (-files), cheaper than a
    reduce-side join."""
import sys, os, datetime


def load_genres(path=os.environ.get("MOVIES_FILE", "movies.dat")):
    d = {}
    for line in open(path, encoding="utf-8", errors="ignore"):
        p = line.rstrip("\n").split("::")
        if len(p) >= 2:
            d[p[0]] = p[2].split("|") if len(p) > 2 and p[2] else ["Unknown"]   # edge case:
                missing genre
    return d


def mapper(job):
    genres = load_genres() if "genre" in job else None
    for line in sys.stdin:
        p = line.rstrip("\n").split("::")
        try: user, movie, rating, ts = p[0], p[1], int(p[2]), int(p[3])
        except (ValueError, IndexError): continue                                  # edge case:
            malformed row
        if job == "movie":   keys = [movie]
        elif job == "user":  keys = [user]
        elif job == "month": keys = [datetime.datetime.fromtimestamp(ts,
            datetime.timezone.utc).strftime("%Y-%m")]
        else:
            g = genres.get(movie)
            if not g: continue                                                     # movie missing
                from catalogue
            keys = {"genre": g, "genre_rating": [f"{x}|{rating}" for x in g],
                    "user_genre": [f"{user}|{x}" for x in g]}[job]
        for k in keys: print(f"{k}\t1\t{rating}")


def reducer():
    cur, c, s = None, 0, 0
    for line in sys.stdin:                       # input arrives sorted by key
        k, n, r = line.rstrip("\n").split("\t")
        if k != cur:
            if cur is not None: print(f"{cur}\t{c}\t{s}")
            cur, c, s = k, 0, 0
        c += int(n); s += int(r)
    if cur is not None: print(f"{cur}\t{c}\t{s}")


if __name__ == "__main__":
    mapper(sys.argv[2]) if sys.argv[1] == "map" else reducer()
```

Complexity. The mapper is linear in the number of ratings. The sort between map and reduce is O(n log n) in the number of intermediate records, and the reducer is a single linear pass because its input arrives sorted by key. Memory use is O(1) in the reducer and O(number of movies) in the map-side join dictionary.

3.6 Apache Spark Implementation

The same six aggregations are written with the Spark DataFrame API. The ratings and movies files are parsed from text, the genre column is exploded into one row per genre, and the small movie table is joined to the large ratings table using a **broadcast join**, which is the DataFrame equivalent

of the map-side join. Malformed rows are removed with dropna, and movies without a genre receive “Unknown”, matching the MapReduce behaviour exactly.

CODE · spark/spark_analysis.py

```
"""PySpark implementation of the same six aggregations (DataFrame API). Usage: spark_analysis.py
    <raw_dir> <out_dir>"""
import sys, os
from pyspark.sql import SparkSession, functions as F
src, out = sys.argv[1], sys.argv[2]; os.makedirs(out, exist_ok=True)
sp = (SparkSession.builder.master("local[*]").appName("MovieInsights")
      .config("spark.ui.enabled", "false").config("spark.sql.session.timeZone",
          "UTC").getOrCreate())
sp.sparkContext.setLogLevel("ERROR")
p = F.split("value", "::")
r = (sp.read.text(f"{src}/ratings.dat").select(p.alias("p"))
       .select(F.col("p")[0].alias("user"), F.col("p")[1].alias("movie"),
               F.col("p")[2].cast("int").alias("rating"), F.col("p")[3].cast("long").alias("ts"))
       .dropna())                                              # drop malformed rows
m = (sp.read.text(f"{src}/movies.dat").select(p.alias("p"))
       .select(F.col("p")[0].alias("movie"),
               F.when((F.size("p") > 2) & (F.length(F.col("p")[2]) > 0),
                   F.col("p")[2]).otherwise("Unknown").alias("g"))
       .select("movie", F.explode(F.split("g", "\\|")).alias("genre")))
j = r.join(F.broadcast(m), "movie")                            # broadcast = map-side join
specs = {"movie": r.groupBy(F.col("movie").alias("k")), "user":
    r.groupBy(F.col("user").alias("k")),
         "month": r.groupBy(F.date_format(F.from_unixtime("ts"), "yyyy-MM").alias("k")),
         "genre": j.groupBy(F.col("genre").alias("k")),
         "genre_rating": j.groupBy(F.concat_ws("|", "genre", "rating").alias("k")),
         "user_genre": j.groupBy(F.concat_ws("|", "user", "genre").alias("k"))}
for name, g in specs.items():
    with open(f"{out}/{name}.tsv", "w") as f:
        for k, c, s in g.agg(F.count("*"), F.sum("rating")).collect(): f.write(f"{k}\t{c}\t{s}\n")
sp.stop()
```

3.7 Orchestration and Cross-Validation

run_pipeline.py runs every MapReduce job through the same map | sort | reduce sequence that Hadoop Streaming applies, then runs the Spark job, and compares the two outputs key by key. A job is accepted only if both engines produce identical (count, sum) pairs for every key. It then derives the dashboard feed: weighted ratings, favourite genres, rating histograms, monthly series and the benchmark figures.

CODE · validation step in run_pipeline.py

```
mr = {j: load(f"{OUT}/mapreduce/{j}.tsv") for j in JOBS}
sp = {j: load(f"{OUT}/spark/{j}.tsv") for j in JOBS}
match = {j: mr[j] == sp[j] for j in JOBS}
```

3.8 Analytical Methods

- **Average rating** of a key: sum of ratings divided by number of ratings.
- **Weighted rating** for ranking movies, so a title with few ratings cannot top the list: WR = (v / (v + m)) × R + (m / (v + m)) × C, where v is the movie’s rating count, R its mean rating, C the global mean (7.31) and m = 100 is the prior weight. Only movies with at least 200 ratings are ranked.
- **Active users** are ranked by the number of ratings they gave.

- **Genre preference** is measured in two ways: the rating histogram of each genre compared with all genres, and the share of ratings of 8 or more ("like rate").
- **Favourite genre** of a user is the genre in which the user gave the most ratings (computed for users with 20 or more ratings).

3.9 Edge Cases Handled

Situation	Handling
Malformed line or non-numeric rating	Skipped by the mapper (try/except) and dropped by dropna in Spark
Movie without genre	Genre set to "Unknown"
Movie missing from the catalogue	Row ignored in genre jobs, counted in movie job
Genres with very few ratings (for example Game-Show with 1)	Dashboard offers a minimum-ratings slider; written findings use genres with 20,000+ ratings
Movies with very few ratings	Weighted rating plus a 200-rating minimum
Suspicious rating patterns	Flagged in the analysis (Section 4.3) rather than silently removed
Months with very low activity	Hidden from the trend chart (fewer than 200 ratings)

Results and Analysis

4.1 Overview

Measure	Value
Ratings analysed	921,398
Distinct users	71,707
Distinct movies rated	38,018
Mean rating (0 to 10)	7.31
Heavy users (20+ ratings)	9,303 (13.0% of users)
Share of ratings held by the top 1% of movies	38.8%
Genre assignments (rating × genre)	2,803,395

4.2 Most-Rated Movies

Rating volume is used as a measure of popularity. The ranking is dominated by widely discussed releases from 2013 to 2015 and 2019, which reflects when the underlying tweets were most numerous (Section 4.7).

#	Title	Ratings	Mean rating
1	Gravity (2013)	3,104	8.23
2	Interstellar (2014)	2,964	8.85
3	1917 (2019)	2,893	8.62
4	The Wolf of Wall Street (2013)	2,842	8.39
5	Joker (2019)	2,757	9.08
6	Man of Steel (2013)	2,699	7.66
7	World War Z (2013)	2,431	7.3
8	Iron Man Three (2013)	2,425	7.67
9	Now You See Me (2013)	2,380	7.59
10	Gone Girl (2014)	2,304	8.23

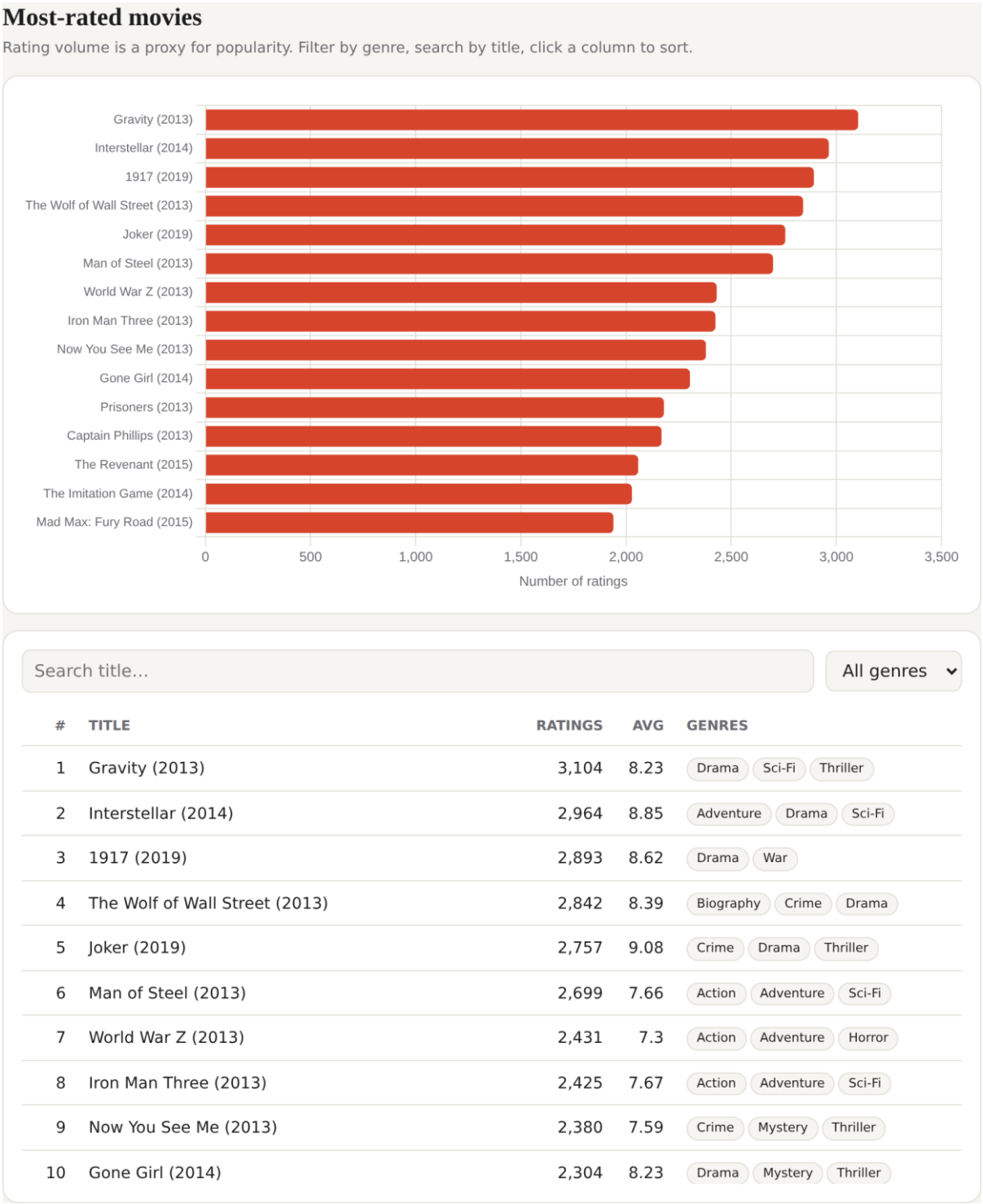


Figure 4. Dashboard view: most-rated movies chart and searchable, sortable table.

Popularity and quality are different things: Gravity (2013) has the most ratings (3,104) with a mean of 8.23, while The Shawshank Redemption (1994) has far fewer ratings (1,177) but a mean of 9.39.

4.3 Highest-Rated Movies

#	Title	Weighted	Mean	Ratings
1	Be Somebody (2016)	9.396	9.99	353
2	The Shawshank Redemption (1994)	9.226	9.39	1,177
3	The Dark Knight (2008)	9.046	9.28	740
4	Joker (2019)	9.02	9.08	2,757
5	12 Angry Men (1957)	9.016	9.21	874
6	Avengers: Endgame (2019)	8.937	9.05	1,474
7	The Lord of the Rings: The Return of the King (2003)	8.933	9.35	392
8	Inception (2010)	8.877	9.02	1,056

Data-quality finding. Be Somebody (2016) has a mean rating of 9.99 over 353 ratings, which is far outside what any real film achieves; it ranks first only because of this pattern. It is almost certainly the result of coordinated or automated rating. The pipeline keeps it visible and flags it, because deciding whether to exclude such titles is an analysis decision that should be documented, not hidden. Excluding it, the list is headed by the classic titles that audiences rate consistently highly.

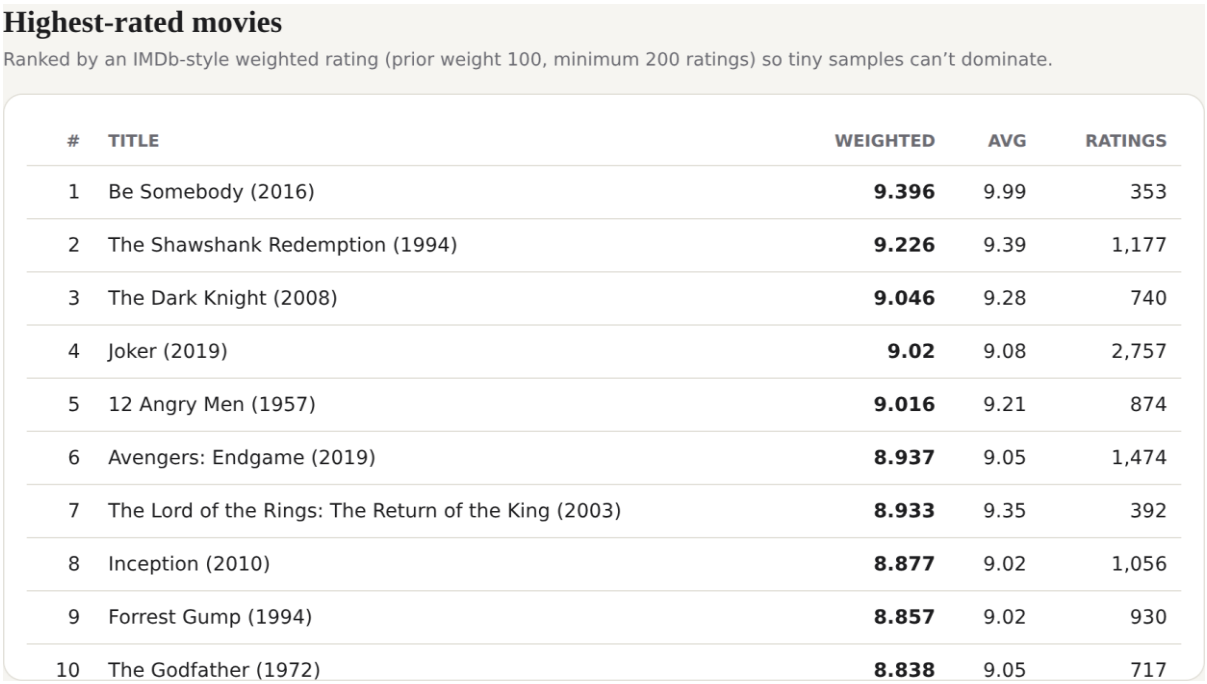


Figure 5. Dashboard view: highest-rated movies by weighted rating.

4.4 Highest-Rated Genres

Genres with fewer than 20,000 ratings (for example News, Reality-TV, Game-Show and Adult) are excluded from the written comparison because their means are driven by very few ratings.

Genre	Ratings	Mean rating
Biography	91,982	7.83
Music	28,580	7.8
War	45,819	7.73
History	39,656	7.71
Animation	42,093	7.69
Sport	22,493	7.67
Drama	513,794	7.53
Family	59,322	7.5
Adventure	232,067	7.34
Crime	185,404	7.33
Romance	126,902	7.32
Fantasy	112,679	7.28
Sci-Fi	184,051	7.28
Mystery	117,460	7.23
Comedy	243,211	7.15
Action	290,680	7.13
Thriller	320,474	7.12
Horror	93,021	6.5

Biography (7.83), **Music** (7.8) and **War** (7.73) have the highest means, while **Horror** (6.5) is clearly the lowest, about 1.33 points below the best genre. Niche and fact-based genres tend to score higher, probably because viewers choose them deliberately, while genres with mass appeal and heavy volume such as Thriller and Action score lower. This is a selection effect in the data, not proof that one genre is “better”.

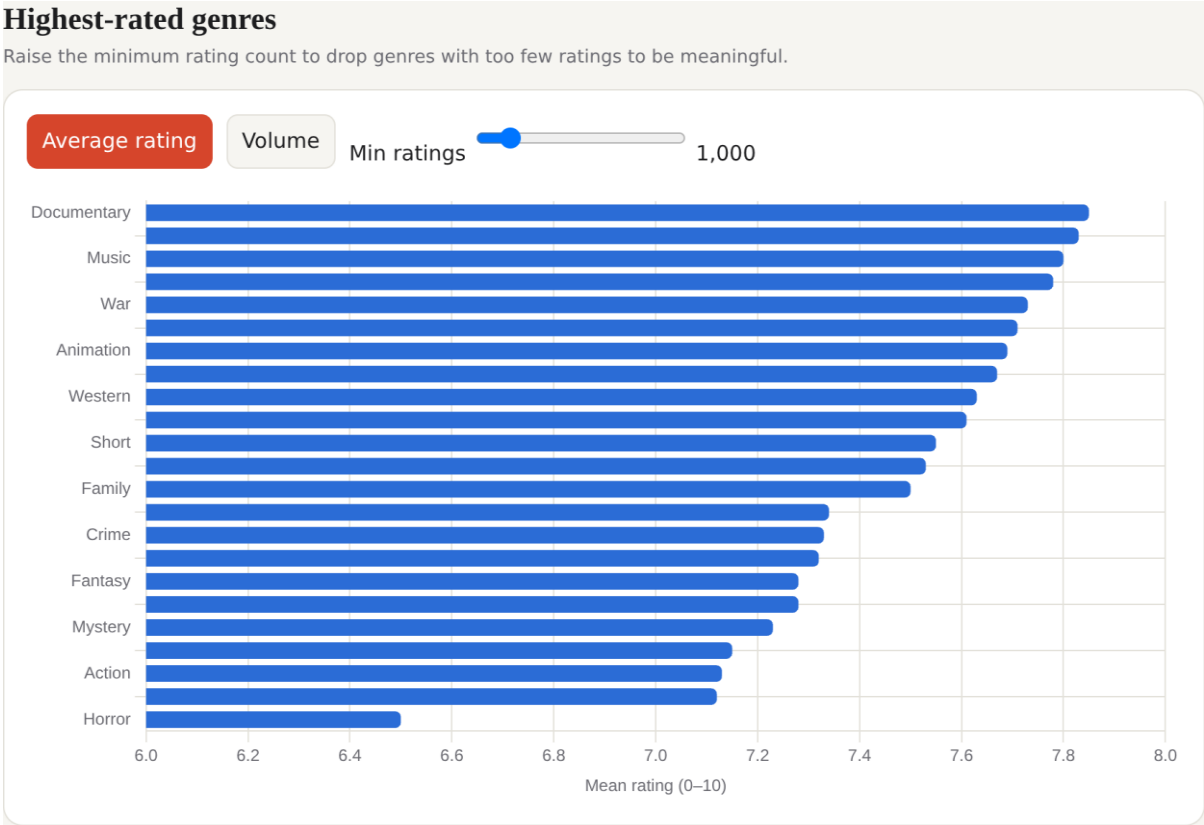


Figure 6. Dashboard view: genres ranked by mean rating, with the minimum-ratings slider.

4.5 Most Active Users

#	User ID	Ratings	Mean rating	Favourite genre
1	17405	2,875	6.23	Drama
2	26962	2,070	6.74	Drama
3	40861	1,924	6.93	Drama
4	69530	1,770	6.42	Drama
5	69827	1,606	4.6	Thriller
6	2216	1,377	6.98	Horror
7	71586	1,369	6.72	Horror
8	3664	1,274	7.32	Drama

The most active user rated 2,875 movies. The 25 most active users together gave 31,077 ratings, which is 3.4% of all ratings. User activity is highly skewed: the median user rated only 2 movies, while 9,303 users (13.0%) rated at least 20. Active users also differ in strictness; the mean ratings above range from 4.6 to 7.32.

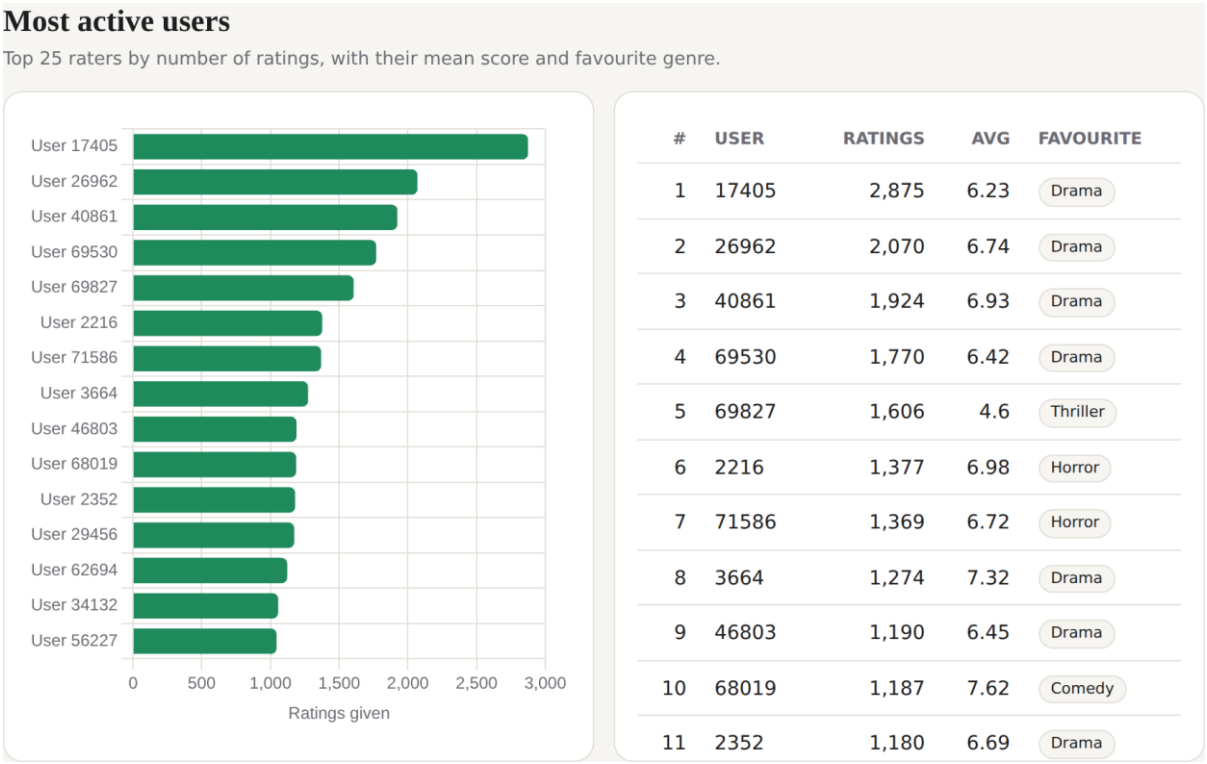


Figure 7. Dashboard view: most active users with mean rating and favourite genre.

4.6 Genre-Wise Preferences

Two views of preference were computed. First, the share of ratings that are 8 or higher ("like rate") and 4 or lower ("dislike rate") per genre, compared with all genres. Second, the favourite genre of heavy users.

Genre	Like rate (8+)	Dislike rate (≤4)
All genres (per genre assignment)	50.1%	6.7%
Documentary	63.4%	3.0%
Biography	63.1%	2.6%
Drama	55.7%	5.3%
Animation	58.2%	3.6%
Action	44.8%	7.9%
Horror	32.2%	14.5%

Horror has the lowest like rate and the highest dislike rate among the genres shown, which matches its lowest mean rating; Documentary and Biography have the highest like rates. For favourite genres, **Drama** dominates: 6,245 of the 9,303 heavy users (67%) rate Drama most often, followed by Action and Thriller.

Favourite genre	Heavy users	Share
Drama	6,245	67.1%
Action	1,579	17.0%
Thriller	600	6.4%
Comedy	366	3.9%
Adventure	357	3.8%
Horror	91	1.0%
Sci-Fi	33	0.4%
Romance	11	0.1%

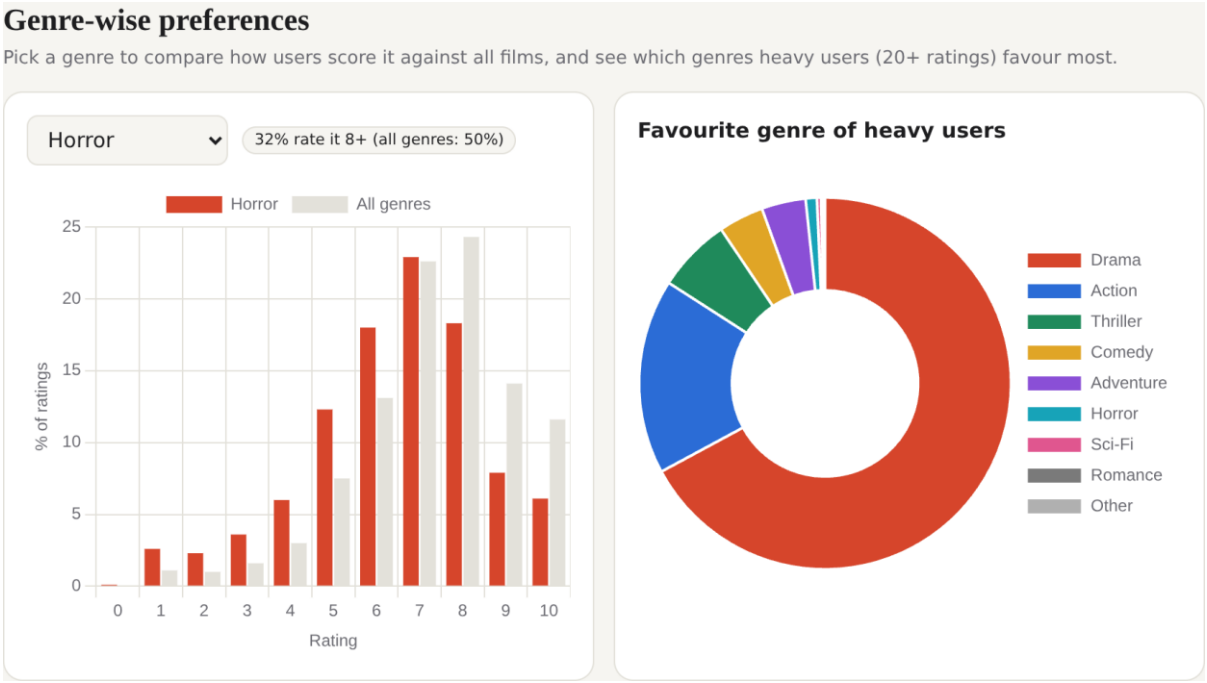


Figure 8. Dashboard view: rating distribution of a selected genre against all genres, and favourite genres of heavy users.

4.7 Rating Activity over Time

Year	Ratings	Mean rating
2013	170,807	7.30
2014	156,354	7.28
2015	123,644	7.30
2016	112,455	7.31
2017	105,987	7.31
2018	81,870	7.29
2019	74,183	7.38
2020	63,115	7.44
2021	32,983	7.20

Rating volume peaks in 2013 and declines afterwards, while the mean rating stays within a narrow band. This explains why the most-rated list is concentrated on films released in 2013 to 2015: the largest number of ratings were posted in those years, mostly for contemporary releases.

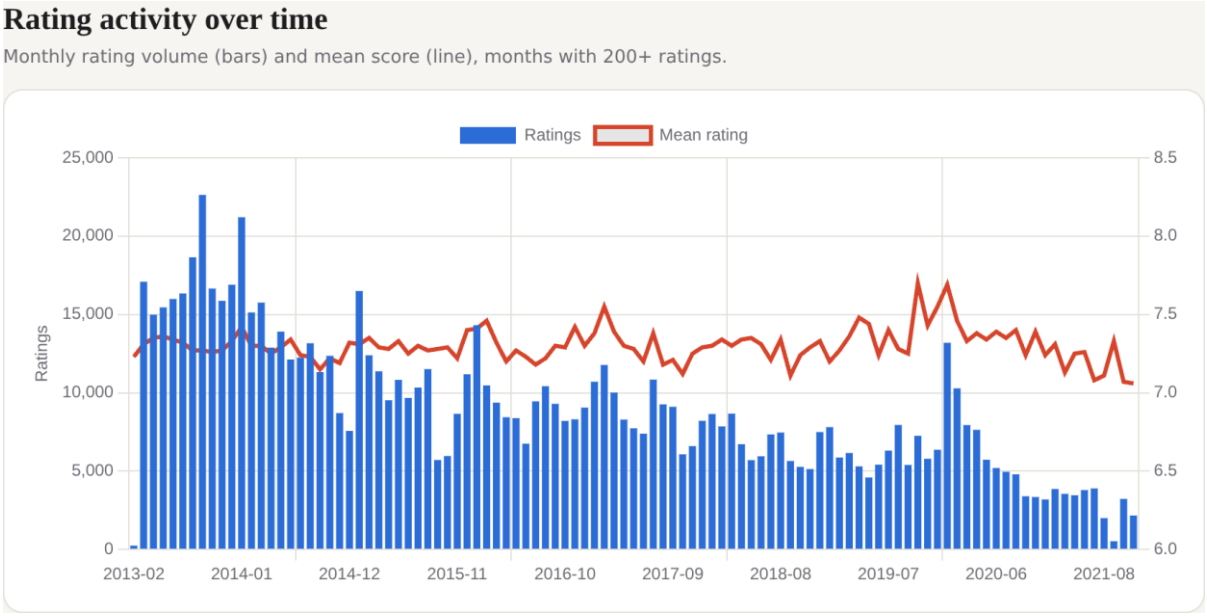


Figure 9. Dashboard view: monthly rating volume and mean rating.

4.8 Performance Evaluation

Performance was measured on a single node. The goals were to check correctness across engines, find which jobs are expensive, test how run time grows with data size, and measure how much a combiner reduces shuffle volume.

Correctness across engines

OUTPUT · python run_pipeline.py

```
MR time 68.09 s   Spark time 53.14 s
outputs identical: movie: True, genre: True, user: True, genre_rating: True, user_genre: True,
    month: True
```

All six jobs gave identical results in MapReduce and Spark for every key, which is strong evidence that both implementations are correct.

Engine and job comparison

Engine	Jobs	Time (s)	Notes
Hadoop Streaming MapReduce (local run)	6	68.09	Re-reads and re-sorts the input for every job
Apache Spark (local[*])	6	53.14	One session, broadcast join, includes JVM start-up
pandas baseline (single process)	3 (movie, user, month)	5.04	In memory, aggregation only, excludes the file read

Spark finished the six jobs about 22% faster than the MapReduce run, mainly because it reads and parses the data once for all jobs while the streaming run repeats the work per job. The in-memory pandas baseline is faster than both for the three jobs it covers; this is expected for data that fits in memory on one machine and is the reason distributed engines are chosen only when data or processing no longer fits on a single machine. The value of the MapReduce and Spark versions here is that they scale out unchanged.

MapReduce job	Time (s)
movie	4.89
genre	15.43
user	4.75
genre_rating	15.52
user_genre	16.37
month	8.45
total (profiling run)	**65.4**

The genre-based jobs cost about three times as much as the movie and user jobs because every rating is expanded into one record per genre (2,803,395 intermediate records instead of 921,398).

Scalability

Share of ratings	Rows processed	Time (s), genre job
25%	230,349	3.83
50%	460,699	7.96
75%	691,048	12.11
100%	921,398	16.14

Run time grows almost exactly linearly with input size (3.83 s at 25% to 16.14 s at 100%), which matches the O(n) map and the sort-dominated shuffle. This predictable behaviour is what allows capacity to be planned: on a cluster, map tasks run in parallel over blocks, so the same linear cost is divided across workers.



Figure 10. Per-job run time, scaling with data size, and engine comparison.

Combiner effect

Job	Map output records	Distinct keys	Reduction at best
movie	921,398	38,018	24.2x
genre	2,803,395	29	96,669x
user	921,398	71,707	12.8x
genre_rating	2,803,395	284	9,871x

Job	Map output records	Distinct keys	Reduction at best
user_genre	2,803,395	514,743	5.4x
month	921,398	103	8,946x

A combiner collapses the map output of each input split to at most one record per distinct key, so the shuffle carries up to the factor shown in the last column fewer records. The effect is largest for the genre-level jobs, where only 29 genres exist.

4.9 Inference

- **Volume is not quality.** The most-rated movies are not the highest-rated, so recommendation by popularity alone and by mean rating alone would give different lists.
- **Extreme sparsity.** The user-movie matrix is 99.966% empty, 67% of movies have fewer than 5 ratings and the median user rated 2 films. Any recommender built on this data needs item-based or content-based methods and a fallback for new users.
- **Heavy concentration.** The top 1% of movies hold 38.8% of all ratings: a classic long-tail distribution.
- **Selection effects.** Because ratings are self-selected shares, niche genres rate higher and the overall mean is positive (7.31). Genre rankings should be read as patterns among these users.
- **Data quality matters.** One suspicious title would have topped a naive ranking; the weighted rating and a manual flag made the issue visible.

Dashboard Application

The results are presented in a single-page web dashboard, dashboard/index.html. It is a self-contained file: the aggregated results (about 50 KB of JSON) are embedded in the page, charts are drawn with Chart.js, and there is no server or build step, so it can be opened locally, hosted on any static server, or shared as a link.

5.1 Features

Section	Interaction
Overview	Headline counts and automatically generated key findings
Most rated	Bar chart of the top 15; searchable table of 300 movies; genre filter; click-to-sort columns
Highest rated	Weighted-rating ranking with rating counts
Genres	Toggle between mean rating and volume; minimum-ratings slider
Active users	Top 25 raters with mean rating and favourite genre
Preferences	Genre selector comparing its rating distribution with all genres, like rate, and favourite-genre chart
Trend	Monthly volume bars with mean-rating line
Pipeline	Architecture flow, engine benchmark and cross-validation badge
Theme	Light and dark theme, following the system setting with a manual toggle

5.2 Landing View and Overview

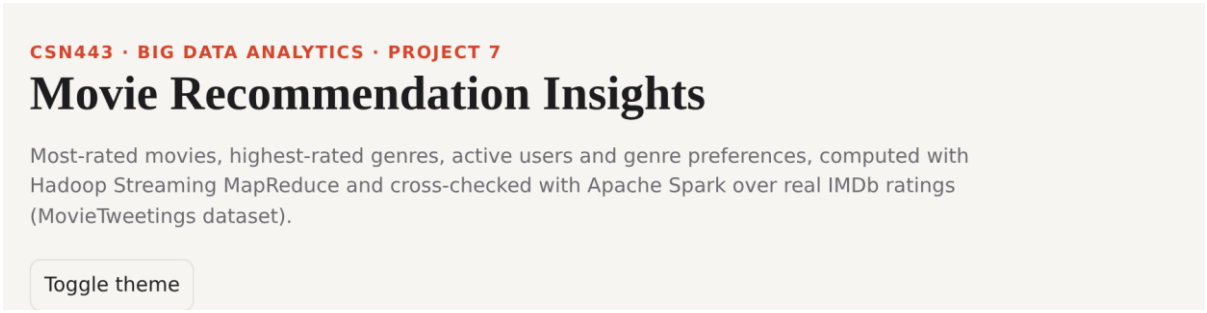


Figure 11. Dashboard landing view with navigation bar.

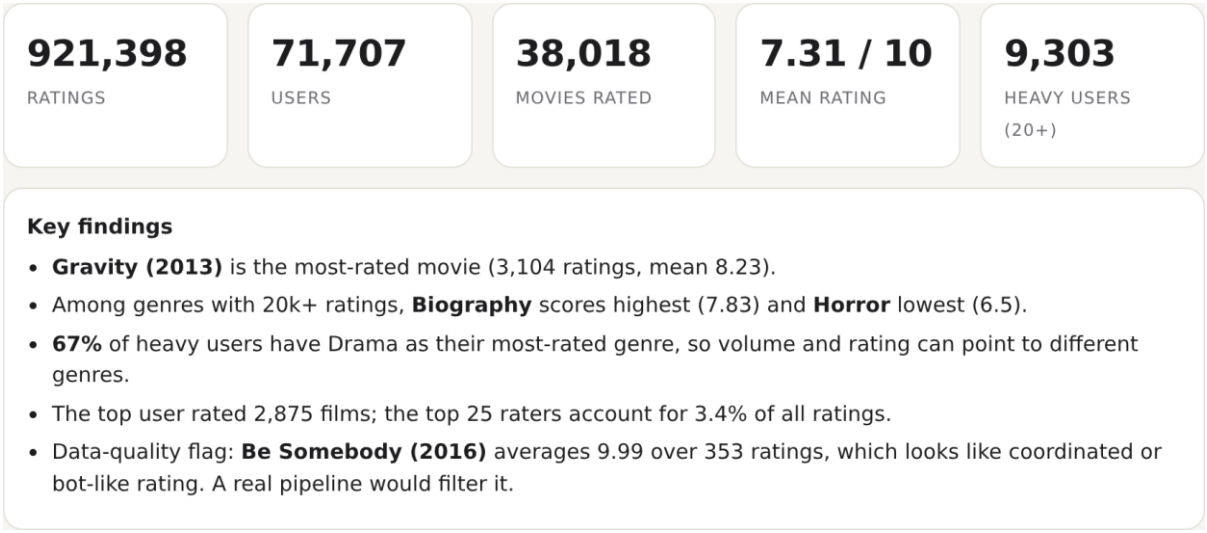


Figure 12. Overview section: headline counts and generated key findings.

5.3 Pipeline and Benchmark Panel

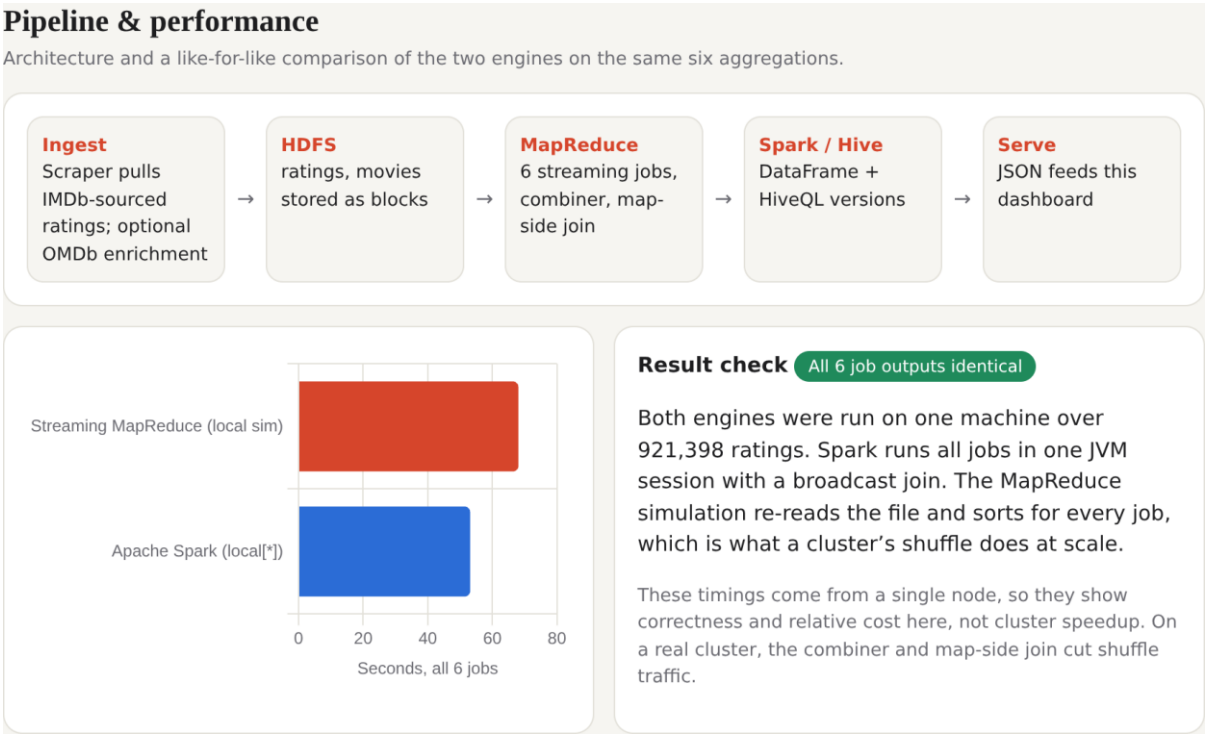


Figure 13. Pipeline panel with architecture flow, benchmark chart and cross-validation result.

5.4 Responsive Design

The layout uses CSS grid with fluid columns, scrollable tables and a horizontally scrollable navigation bar, so the same page works on a phone, a tablet and a desktop. Colours are defined as design tokens so that the light and dark themes stay consistent, and the page respects the safe areas of notched phones.

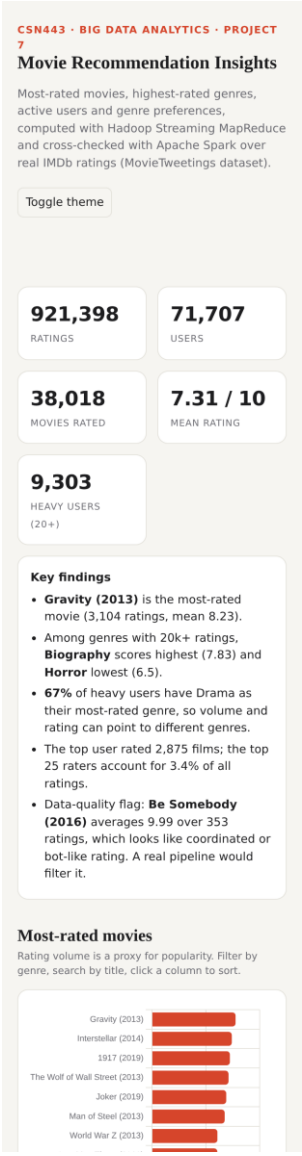


Figure 14. Dashboard on a phone-sized screen (390 px wide).

5.5 Running the Project

CODE · commands

```
pip install pyspark                # Java 11+ required
python scraper/fetch_data.py       # extraction (add --enrich 200 with OMDB_KEY set)
python run_pipeline.py             # MapReduce + Spark + validation + dashboard feed
python load_db.py                  # relational result store
./run_hadoop.sh                    # same jobs on a Hadoop cluster (HADOOP_HOME set)
hive -f hive/queries.hql           # HiveQL version
# open dashboard/index.html in a browser
```

Conclusion and Future Scope

This project implements a complete big data workflow for movie rating analytics. It extracts real IMDb-sourced ratings through a repeatable script, profiles data quality, lays the data out for HDFS and Hive, computes six families of aggregates with Hadoop Streaming MapReduce, reproduces them in Apache Spark with identical output, stores the results in a relational database, measures performance, and presents everything in an interactive, responsive dashboard.

The analysis found that popularity and quality diverge (Gravity (2013) is the most-rated movie but not among the best rated), that Biography is the highest-rated genre and Horror the lowest among large genres, that a small group of users produces a large share of ratings, that Drama is the favourite genre of 67% of heavy users, and that the data is extremely sparse and skewed. It also surfaced a data-quality anomaly that a simple average would have hidden.

6.1 Limitations

- The ratings are self-selected shares from a social platform and are not a random sample of all viewers.
- The dataset contains no demographic or content features, so demographic segmentation and content-based analysis are not possible.
- Timings come from a single node; cluster speedups, HDFS block behaviour and Hive execution time were not measured here, although the scripts for them are provided.
- The OMDb enrichment step needs a personal API key and was not used for the reported results.
- The pipeline is batch-oriented; a new snapshot requires a re-run.

6.2 Future Scope

- Deploy on a multi-node Hadoop or Spark cluster (or a managed service) and measure real speed-up and shuffle volumes.
- Build a recommender with Spark MLlib (ALS) and item-based filtering, and evaluate it with ranking metrics such as precision, recall and NDCG.
- Move ingestion to streaming with Kafka and Spark Structured Streaming to cut latency from batch to near real time.
- Store enriched metadata in HBase or Cassandra and schedule the pipeline with Apache Airflow.
- Add anomaly detection for suspicious rating patterns and sentiment analysis of tweet text.

Bibliography

1. S. Dooms, T. De Pessemier and L. Martens, “MovieTweetings: a Movie Rating Dataset Collected From Twitter”, Workshop on Crowdsourcing and Human Computation for Recommender Systems (CrowdRec) at ACM RecSys, 2013. Dataset: https://github.com/sidooms/MovieTweetings
2. J. Dean and S. Ghemawat, “MapReduce: Simplified Data Processing on Large Clusters”, OSDI, 2004.
3. M. Zaharia et al., “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing”, NSDI, 2012.
4. T. White, Hadoop: The Definitive Guide, O’Reilly Media.
5. Apache Hadoop documentation, including Hadoop Streaming: https://hadoop.apache.org/docs/stable/
6. Apache Spark documentation (PySpark and SQL guide): https://spark.apache.org/docs/latest/
7. Apache Hive language manual: https://cwiki.apache.org/confluence/display/Hive/LanguageManual
8. OMDb API documentation: https://www.omdbapi.com/
9. IMDb non-commercial datasets: https://developer.imdb.com/non-commercial-datasets/
10. Chart.js documentation: https://www.chartjs.org/docs/latest/
11. SQLite documentation: https://www.sqlite.org/docs.html

Dataset note: ratings come from the public MovieTweetings dataset, which gathers ratings that users published through the IMDb apps.

Appendix A. Project Structure and Run Guide

PROJECT STRUCTURE

```
movie-insights/
├── scraper/fetch_data.py        # extraction (+ optional OMDb enrichment)
├── mapreduce/mr_jobs.py         # Hadoop Streaming mapper, reducer, combiner
├── spark/spark_analysis.py      # PySpark implementation
├── hive/queries.hql             # HiveQL tables, views and queries
├── run_hadoop.sh                # HDFS upload and streaming job submission
├── run_pipeline.py              # run, cross-validate, benchmark, build dashboard
├── load_db.py                   # result store (SQLite)
├── dashboard/index.html         # interactive dashboard
├── data/raw/                    # ratings.dat, movies.dat, users.dat
└── results/                     # job outputs, JSON feed, database
```

Sample of a MapReduce output file (key, count, sum of ratings) for the genre job:

OUTPUT · results/mapreduce/genre.tsv (first rows)

```
genre   count    sum_of_ratings
Action  290680   2073276
Adult   13       88
Adventure        232067   1702843
Animation        42093    323660
Biography        91982    719785
Comedy  243211   1739030
```